# Chinook Digital Media Store Database

Dataset-Specific Laboratory Workbook

Data Acquisition & Manipulation with Pandas

**GROUP** Group C • Source: Chinook sample database — SQLite / PostgreSQL / MySQL etc.

## How to use this workbook

You are a data analyst for a digital media store. Unlike flat-file exercises, data lives in normalized relational tables. You must decide what to filter in SQL versus what to manipulate in Pandas.

This workbook assumes Python essentials and notebook use. Its focus is not Python syntax revision; it is disciplined Pandas reasoning: acquire → inspect → diagnose → select/filter → clean/transform → group/aggregate → merge/reconcile → export.

**ROW GRAIN** Invoice is one row per invoice; InvoiceLine is one row per purchased line; Customer is one row per customer; Track is one row per track. Joins change grain.

**KEYS & IDENTITY** CustomerId, InvoiceId, TrackId, AlbumId and GenreId are relational keys. Join validation should reflect one-to-many relationships.

## Learning outcomes

- Acquire the dataset using the source-appropriate Pandas/API/database syntax.
- Diagnose structure, dtypes, missingness, duplicates, ranges, categories and memory implications before cleaning.
- Use loc/iloc, boolean masks, query, isin, between, sort_values and column selection intentionally.
- Clean and transform without overwriting raw data or silently changing analytical meaning.
- Use groupby, agg, named aggregation, value_counts, pivot/crosstab/transform where justified.
- Join/merge/concat with explicit key/cardinality reasoning and post-operation validation.
- Export reproducible outputs and explain why each transformation was necessary.

## Acquisition starter

```
import sqlite3
import pandas as pd

con = sqlite3.connect("Chinook_Sqlite.sqlite")

tables = pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;", con
)
display(tables)

customers = pd.read_sql_query("SELECT * FROM Customer;", con)
invoices = pd.read_sql_query("SELECT * FROM Invoice;", con)
```

```
lines = pd.read_sql_query("SELECT * FROM InvoiceLine;", con)
```

## Core Pandas syntax map

| Capability | Methods / properties |
|---|---|
| **Inspect** | shape, head(), tail(), sample(), columns, dtypes, info(), memory_usage(), describe(), nunique(), value_counts(), isna(), duplicated() |
| **Select** | df[col], df[[...]], loc[rows, cols], iloc[rows, cols], filter(), select_dtypes() |
| **Filter** | boolean masks with & \| ~, isin(), between(), str methods, query(), notna()/isna() |
| **Clean** | copy(), dropna(), fillna(), drop_duplicates(), astype(), to_numeric(), to_datetime(), str.strip(), replace(), map(), rename() |
| **Transform** | assign(), where(), mask(), cut(), qcut(), apply()/map() when justified; prefer vectorization |
| **Group** | groupby(), size(), count(), sum(), mean(), median(), min(), max(), nunique(), agg(), transform(), pivot_table(), crosstab() |
| **Combine** | merge(how=..., on=..., validate=..., indicator=True), join(), concat(), suffixes |
| **Validate/export** | assert, equals(), compare counts/totals, to_csv(index=False), to_excel(index=False), to_parquet() |

## Case Study 1: Database acquisition & source-side filtering

**REAL-LIFE QUESTION** When should SQL reduce rows/columns before Pandas receives them?

**WHY THIS MATTERS** A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

1. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
2. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
3. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
4. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
5. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
6. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

### Guided implementation

```
q = '''
SELECT InvoiceId, CustomerId, InvoiceDate, BillingCountry, Total
FROM Invoice
WHERE Total > 5
ORDER BY Total DESC
'''
large_invoices = pd.read_sql_query(q, con, parse_dates=["InvoiceDate"])
large_invoices.info()
large_invoices.head()
```

### Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

### Checkpoint evidence

| Evidence | Your result | Interpretation |
|---|---|---|
| Shape / row count | _____ | _____ |
| Missing / duplicate check | _____ | _____ |
| Key or total reconciliation | _____ | _____ |

### Practice ladder

**Basic —** Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

**Intermediate —** Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

**Challenge —** Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

**Debug —** Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

## Case Study 2: Inspect/select relational tables

**REAL-LIFE QUESTION** What is the grain and key uniqueness of each table?

**WHY THIS MATTERS** A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

7. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
8. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
9. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
10. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
11. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
12. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

### Guided implementation

```
for name, frame, key in [
    ("Customer",customers,"CustomerId"),
    ("Invoice",invoices,"InvoiceId"),
    ("InvoiceLine",lines,"InvoiceLineId")
]:
    print(name, frame.shape, "key unique:", frame[key].is_unique,
          "duplicates:", frame.duplicated().sum())
    print(frame.isna().sum().sort_values(ascending=False).head())
```

### Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

### Checkpoint evidence

| Evidence | Your result | Interpretation |
|---|---|---|
| Shape / row count | _____ | _____ |
| Missing / duplicate check | _____ | _____ |
| Key or total reconciliation | _____ | _____ |

### Practice ladder

**Basic —** Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

**Intermediate —** Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

**Challenge —** Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

**Debug —** Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

## Case Study 3: Clean & transform invoice data

**REAL-LIFE QUESTION** How do you preserve invoice IDs while deriving date features and customer value summaries?

**WHY THIS MATTERS** A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

13. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
14. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
15. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
16. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
17. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
18. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

### Guided implementation

```
inv=invoices.copy()
inv["InvoiceDate"]=pd.to_datetime(inv["InvoiceDate"], errors="coerce")
inv["invoice_year"]=inv["InvoiceDate"].dt.year
inv["invoice_month"]=inv["InvoiceDate"].dt.to_period("M").astype(str)
inv["Total"]=pd.to_numeric(inv["Total"], errors="coerce")
assert inv["InvoiceId"].is_unique
```

### Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?

- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

### Checkpoint evidence

| Evidence | Your result | Interpretation |
|---|---|---|
| Shape / row count | _____ | _____ |
| Missing / duplicate check | _____ | _____ |
| Key or total reconciliation | _____ | _____ |

### Practice ladder

**Basic —** Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

**Intermediate —** Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

**Challenge —** Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

**Debug —** Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

## Case Study 4: Group & aggregate customer/country KPIs

**REAL-LIFE QUESTION** Which countries and customers generate the most invoice revenue?

**WHY THIS MATTERS** A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

19. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
20. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
21. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
22. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
23. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
24. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

### Guided implementation

```
country = (
    inv.groupby("BillingCountry", dropna=False)
       .agg(invoices=("InvoiceId","nunique"),
            customers=("CustomerId","nunique"),
            revenue=("Total","sum"),
            avg_invoice=("Total","mean"))
       .sort_values("revenue", ascending=False)
       .reset_index()
)
customer = inv.groupby("CustomerId").agg(
    invoices=("InvoiceId","nunique"), revenue=("Total","sum")
).sort_values("revenue", ascending=False)
```

### Stop & Predict

- Which object(s) will change shape after this code?

- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

### Checkpoint evidence

| Evidence | Your result | Interpretation |
|---|---|---|
| Shape / row count | _____ | _____ |
| Missing / duplicate check | _____ | _____ |
| Key or total reconciliation | _____ | _____ |

### Practice ladder

**Basic —** Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

**Intermediate —** Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

**Challenge —** Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

**Debug —** Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

## Case Study 5: Multi-table merge challenge

**REAL-LIFE QUESTION** Which genres generate sales, and how do you avoid confusing line revenue with invoice Total?

**WHY THIS MATTERS** A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

25. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
26. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
27. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
28. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
29. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
30. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

### Guided implementation

```
tracks=pd.read_sql_query("SELECT TrackId, Name, AlbumId, GenreId, UnitPrice FROM Track;", con)
genres=pd.read_sql_query("SELECT GenreId, Name AS GenreName FROM Genre;", con)

assert tracks["TrackId"].is_unique
assert genres["GenreId"].is_unique

x=(lines
   .merge(tracks, on="TrackId", how="left", validate="many_to_one", suffixes=("_sale","_catalog"))
   .merge(genres, on="GenreId", how="left", validate="many_to_one"))
x["line_revenue"]=x["UnitPrice_sale"]*x["Quantity"]

genre_sales=(x.groupby("GenreName", dropna=False)
              .agg(lines=("InvoiceLineId","size"),
```

```
                    units=("Quantity","sum"),
                    revenue=("line_revenue","sum"))
                .sort_values("revenue", ascending=False))
genre_sales.head(10)

con.close()
```

### Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

### Checkpoint evidence

| Evidence | Your result | Interpretation |
|---|---|---|
| Shape / row count | _____ | _____ |
| Missing / duplicate check | _____ | _____ |
| Key or total reconciliation | _____ | _____ |

### Practice ladder

**Basic —** Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

**Intermediate —** Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

**Challenge —** Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

**Debug —** Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

## Final Integrated Case

Management gives you the raw source and asks for a decision-ready table. Build an end-to-end notebook without copying the guided solution blocks. Your notebook must contain:

- source/provenance note and acquisition code
- grain and key statement
- dataset-health table
- documented filtering population
- cleaning decision log
- at least two derived features where meaningful
- at least two grouped summaries
- one merge/concat/reconciliation task appropriate to this dataset
- three validation assertions/checks
- clean export with index=False where CSV/Excel is used
- 200–300 word management note stating findings, exclusions, assumptions and limitations

## Common pitfalls — non-negotiable

| Pitfall | Required response |
|---|---|
| **and / or in Series filters** | Use & / \| with each condition in parentheses. |
| **Chained assignment** | Use .loc[…] = …; do not use df[df['a']>5]['b']=0. |
| **Comparing NaN with ==** | Use .isna() / .notna(). |
| **Modifying a slice** | Use .copy() when creating a working subset. |

| | |
|---|---|
| **Ignoring dtypes** | Run info() early; IDs often should not be numeric measures. |
| **Blind dropna/fillna** | First explain what missing means in that field and question. |
| **iterrows on large data** | Prefer vectorized expressions/groupby/merge. |
| **Unvalidated merge** | Check key uniqueness, use validate= and indicator=True, reconcile rows/totals. |
| **Overwriting raw files** | Keep raw immutable; write clean/derived files separately. |
| **CSV index leakage** | Use to_csv(..., index=False) unless the index is intentionally part of the data. |

## Oral defence prompts

31. What does one row represent?
32. Which columns are identifiers rather than measures?
33. Which missing values did you retain, drop or impute—and why?
34. Show one operation that changed row count. Prove the change was intended.
35. What was the cardinality of your merge? How did you verify it?
36. Which metric would be wrong if you counted rows instead of unique business entities?
37. What would you change if this dataset had 50 million rows?
38. Which part belongs in SQL/API/source filtering rather than Pandas?
39. What conclusion can you NOT make from this dataset?

**END OF DATASET LAB • Preserve raw data • Explain every decision • Validate every join**